



M2Radar — Handoff for Max

What was built on top of the radar, what was taken from Latest, where the system stands, what next

Table of contents

1. Handoff to Max — what happened on the radar, 8–12 September 2026

- a. 1. TL;DR
- b. 2. What was built, by layer
- c. 3. What was taken from `Latest`, what was not, and why
- d. 4. Verdict on the 7-Sep ten-card release and the 11-Sep reconciled report
- e. 5. Corpus and evidence — the denominators, and what they cannot answer
- f. 6. Ten key findings
- g. 7. The ten cards
- h. 8. Where the system stands
- i. 9. Open decisions and questions
- j. 10. How to run things
- k. 11. Security
- l. Appendix — the shortest list of files to read, in order

Handoff to Max — what happened on the radar, 8–12 September 2026

Written 2026-09-12 for Max (`mcharniuk1-spec`). Audience of one, technical. Purpose: you can see exactly what was built on top of the radar this week, what of your work was taken and what was not and why, where the system stands right now, and what to do next. Every number here is read from a file in this repository (paths are repo-relative) or from `data/radar.db` ; where a number could not be reproduced locally it says so on the same line.

Authoritative sources, in order of weight: `reports/M2RADAR_INTEGRATION_EXECUTION_REPORT.md` (the execution record), `reports/audit/01-branch-comparison.md` (the branch audit that decided the merge), `reports/audit/04-existing-cards-review.md` (the verdict on all 33 existing cards), `reports/audit/06-final-qa.md` (independent QA), `engine/SPEC.md` (the engine contract), `docs/M2RADAR_CONTENT_ENGINE_ARCHITECTURE.md` (module map and flow).

1. TL;DR

1. Main's SQLite stays the source of truth. Your `Latest` was adopted as **contracts**, not as a store — state DAG, scene contract, Remotion EDL contract, provider catalog, bounded media workers.
2. Nothing of yours was modified. `origin/Latest` was only fast-forwarded to `6462f409` , which is your fork's `main` ; the fork itself was never written to.
3. A new `engine/` package (30 Python modules) sits on top of the 14 legacy tables, adding 18 versioned tables inside the same `data/radar.db` . Migration deltas were all `+0` — nothing moved, nothing dropped.
4. Coverage is small and stated everywhere: **268 of 3,211 ingested reel codes are analysis-ready (8.3 %)**, and that 268 is the top of `score.py` 's own ranking, not a random sample (median `robust_z` +1.26 vs 0.00).
5. 30 insights → 24 hypotheses → 10 selected → 41 hypothesis-specific references → 10 cards, each with a script reviewed twice (107 findings), a 70-scene storyboard, rendered template frames and a validated Remotion EDL.
6. One real Remotion render was executed end to end: 1080×1920, 30 fps, 60.05 s, 3,433,827 bytes, Remotion 4.0.520, with a hashed receipt.
7. The verdict on your 7-Sep ten-card release is: **the JSON schema is worth keeping, the content is not** — 0 of 10 have a statistically justified reference, 0 quote a real transcript, 0 have real frames.
8. Your 1,062 transcripts and 19,808 frames are **not in git on either branch** (full tree scan). They could not be ported. If you export them, there is an import path waiting.
9. Notion was applied for real on 2026-09-12 from the server: dashboard body rewritten, 7 databases created and filled. `/opt/radar` is on `main` with the engine DB in place, old DB backed up.
10. Next scheduled run — the first one that exercises the new stages — is **Monday 2026-09-14, 07:00 server time**.

2. What was built, by layer

Thirty modules under `engine/` (plus the package marker), plus tests, schemas, docs and a Node render path. `engine/SPEC.md` is the binding contract for all of it;

`docs/M2RADAR_CONTENT_ENGINE_ARCHITECTURE.md` is the readable map. The north-star flow (architecture doc §2):

```
Hiker (paid, weekly)
-> reels/accounts snapshot (db.py, 14 tables)
-> local faster-whisper transcript (engine/local_pipeline.py)
-> local ffmpeg scene/frame extraction (engine/scenes.py)
-> transcript <-> scene alignment (engine/align.py)
-> per-video LLM analysis: ta-v1 beats, fa-v1 frame labels
-> consolidated video_features row (engine/features.py, engine/lexical.py)
-> creator/video statistics, robust z, creator-relative lift (engine/stats.py)
-> numeric insights -> ranked hypotheses
-> hypothesis-specific reference selection (function-tagged)
-> transformed original script + frame-aware storyboard -> card JSON
-> template storyboard frames -> previs EDL (engine/edl.py,
studio/remotion/contract.mjs)
-> [Phase 3] supplied MP4 takes -> bound into the EDL -> Remotion render -> stored
-> Notion projection (engine/notion_sync.py) -> Misha reviews and decides
-> publish (always a separate, explicit, human action)
```

Everything up to "card JSON" runs for free — no paid API beyond the weekly Hiker collection.

2.1 Schema and state tracing

- `engine/schema.py` — versioned, idempotent migration (`schema_migrations`), v1 and v2 applied. `python3 -m engine.schema --check` returns "schema up to date, pending migrations: none".
- `engine/state.py` — runs / jobs / video_state . This is your state DAG, trimmed: `STAGES` is **25 states**, not your 41 (`reports/final/partA/06-state-tracing.md` : "trimmed from his 41-stage DAG ... to the 25 stages this branch actually runs"). `jobs.state` ∈ PENDING | RUNNING | DONE | FAILED | RETRY_REQUIRED | SKIPPED.
- `video_state` is the per-code processing contract: `media_state` , `transcript_state` , `frames_state` , `alignment_state` , `transcript_analysis_state` , `frame_analysis_state` , `features_state` , `analysis_ready` , `corpus_tier` , plus `asr_version` / `frames_version` / `analysis_version` / `features_version` and `flags_json` .
- `engine/migrate_legacy.py` — re-runnable back-fill of the legacy corpus. Run 2026-09-11T22:20:22Z: **every table delta +0** , `integrity_check` ok , 0 foreign-key violations, 0 pk violations. Six unresolved August-archive codes were logged by name, not dropped (`reports/migration-validation.json` | `steps.august_archive.unresolved_codes`).
- Guarded column adds to existing tables only: `reels.fetch_id` , `frames.sha256` , `frames.exists_ok` , `deepdives.evidence_state` , `cards.status_v2` . Nothing renamed, nothing dropped.

2.2 Hiker in git without the key

`reports/audit/03-server-runtime-hiker.md` §1: `git grep -nE "hiker_|HIKER|Bearer|x-access-key"` across `*.py` returns only environment-variable and header **names** — no literal key anywhere. The

committed architecture is `lib/hiker.py` (HTTP client), `engine/hiker_config.py` (config validator, printed by `cron.sh` before every run), `collect_snapshot.py` / `roster.py` / `deep.py` (normalisation into canonical models) and `cron.sh` (13-step chain). Deployable to `/opt/radar` with nothing but a populated `.env`. Server code == `git main`, exactly, at the audit commit.

`engine/providers.py` resolves each provider's state from env presence only, never reading a value. Live probe 2026-09-12: `local-faster-whisper` CONFIGURED (DEFAULT), `local-ffmpeg-scenes` CONFIGURED (DEFAULT), `remotion` CONFIGURED (npx on PATH; Chrome not separately verified), `loore` OPTIONAL_PROVIDER / DISABLED, `supabase` / `cloudflare` / `openai` / `higgsfield` NOT_CONFIGURED. One trap is documented rather than hidden: `lib/hiker.py` also falls back to the local MCP settings file, which the registry deliberately does not inspect, so on such a machine Hiker works while the row says NOT_CONFIGURED.

2.3 Local pipeline and watchdog

`engine/local_pipeline.py::process_video(con, code, url_or_path, run_id, *, asr_version, frames_version)` : download → sha256 → faster-whisper (`small`, int8, CPU, `language='en'`, `vad_filter=True`) → ffmpeg scene detect (`select='gt(scene,0.30)'`, `showinfo`) → scenes with `boundary_reason='detector'` → keyframes + hook samples at 0.4/1.2/2.4/4.0 s (compat with the legacy fixed-9) → contact sheet → alignment → delete the mp4 → `refresh_video_state`. Idempotent, retryable, every step a `jobs` row, versioned by `asr_version` / `frames_version`.

`engine/watchdog.py` is the catch-up worker for ingested-but-not-analysed codes. It replays URLs already sitting in `cache/**/*clips*.json` from a prior paid collection — **it never calls HikerAPI and never pays**. Lock file `data/watchdog.lock` (pid + timestamp, stale after 3 h). `--dry` previews selection without touching disk or the DB. Where no live URL resolves it marks `media_state=EXPIRED`. Ported from your work: `engine/process_budget.py` (bounded subprocess capture, minimal environment, process-group cleanup) wraps the ASR and ffmpeg calls.

2.4 Semantic analysis contracts and the scheduled stage

Two LLM output contracts, one file per code, both defined in `engine/SPEC.md` §5:

- **ta-v1** — `data/analysis/transcripts/<code>.json` : `beats[]` (idx, role, start/end, text, `segment_idx`, `audience_function`, `emotion`, `intention`, `persuasion`, `key_phrases`), `semantics{}` (topic, subtopic, pain, desire, problem, `solution_type`, `proof_type`, `hook_type`, `cta_type`, narrative, tone, `funnel_role`, `positioning_type`, `tools_mentioned`, `numbers_used`, `open_loops`, ...), `interpretation{}`, `quality{}` (1–5 scales), `confidence`.
- **fa-v1** — `data/analysis/frames/<code>.json` : per frame idx → `frame_type`, roll, speaker/face/ui presence, `text_overlay`, `overlay_text`, framing, `dominant_action`, `visual_density`, `is_visual_hook` / `is_cta_visual` / `is_proof_visual`; plus video-level `first_frame_type`, `visual_sequence`, `*_share_est`, `transitions_observed`, `limitations`.

Closed vocabularies live in `engine/SPEC.md` §4 (21 frame types, 13 hook types, 9 CTA types, 13 canonical pains, 10 solution types, 8 proof types, 10 narratives). Out-of-vocabulary labels are kept in `labels_json` and logged rather than dropped — `data/analysis/ingest_warnings.md` records `thesis`, `demo_first`, `story_open`, `contrast`, `comparison`. Note for whoever writes the next prompt version: `demo_first` is a *hook type* leaking into the *beat role* vocabulary.

`engine/ingest_analysis.py` loads those files into `beats` / `frame_labels` / `scenes` and refreshes `video_state`.

`engine/analyze_pending.py` turns what was a hand-run session pass into a cron stage: it selects pending codes straight from the database (no side ledger to drift), exports batch input files in the same shape as the eight hand-run batches, and — with `--yes` and `claude` on PATH — runs one unattended agent per batch, recording a `jobs` row per batch (`stage=TRANSCRIPT_ANALYSIS / FRAME_ANALYSIS` , `prompt_version='ta-v1' / 'fa-v1'`). Then it ingests, re-aligns, rebuilds features and refreshes state. Lock file `data/analyze.lock` . Without `claude` on PATH it prints `NOT_CONFIGURED` , marks the batch SKIPPED and exits 0 — a normal "nothing to do yet", not a failure.

2.5 Stats and features

- `engine/stats.py` — `creator_stats` (median/mean/MAD/IQR/SD/CV of play, rate medians, `posts_per_week`, `outlier_share`, `consistency_score = 1/(1+CV)` , `reliability ∈ SMALL_SAMPLE / CONSISTENT / HIGH_VARIANCE`) and per-video performance: rates with a denominator guard (`play ≥ 100` else NULL), `view_lift = play/creator_median - 1` , `share/save rate lift`, `robust_z` in log space (`(ln(1+play) - median ln) / (1.4826·MAD ln)`), clipped ±5, same convention as `score.py`), `percentile_in_creator` , `outlier_status` .
- `engine/lexical.py` — phrase and word-class features from a transcript, pure text, no LLM call.
- `engine/features.py` — one `video_features` row per code consolidating semantics + visuals + performance. **3,211 rows at features_version = fv-1** ; `creator_stats` 132 rows.
- Outputs: `reports/stats/stats-2026-09-12.json` (802,749 bytes), `reports/data/` (14 files), 18 charts in `reports/charts/` .

One honest wart, recorded in `engine/HANDOFF_NOTES.md` : `video_features` has a row for every ingested code including the 2,914 with neither transcript nor frames, which makes `features_state='DONE'` corpus-wide and is true of the table but misleading as a state.

2.6 Insight → hypothesis → reference → script → review → card → EDL

- `engine/ingest_insights.py` loads `data/analysis/{insights,hypotheses}.json` into `insights / hypotheses / hypothesis_refs` ; rejected and soft-warned records are logged to `data/analysis/ingest_insights_warnings.md` rather than silently accepted.
- **30 insights** (12 RELIABLE, 16 PROBABLE, 2 INSUFFICIENT). Each row carries `statement` , `claim` , `metric` , `n` , `comparison` , `evidence_json` , `supporting_codes_json` , `supporting_creators_json` , `transcript_pattern` , `frame_pattern` , `interpretation` , `implication` , `confidence` , `limitations` — the numbers and their evidence are in the database, not only in prose.
- **24 hypotheses**, scored on 14 weighted keys (`evidence` 0.12 and `positioning` 0.12 carry the most weight, `assets` and `copy_risk` 0.03 the least), **10 SELECTED / 14 REJECTED**. Selected `total_score` 4.16–4.69; best rejected 3.67, so the cut line is 0.49 wide and nowhere close.
- **41 hypothesis_refs** , all bound to a selected hypothesis (H-11 carries 5, the other nine carry 4). Every row records `function` (hook / pain / proof / cta / screen_proof / ...), `reason` , `useful_beat_ids_json` , `useful_scene_ids_json` , `performance_json` , `transformation` . References are per-hypothesis, not a global pool, and the 41 map one-to-one onto the 41 references embedded in the ten card JSONs.
- **Scripts**: written frame-aware against their own hypothesis's references. `script_versions` holds 20 rows (a `writer` row and, where revised, a `reviewer` row).
- **Reviews**: two independent passes — `reports/analysis/10-script-review-A.md` (cards 01/03/05/07/09, 41 findings) and `10-script-review-B.md` (cards 02/04/06/08/10, 66 findings), **107 findings total**. Nine of ten scripts were revised to a new version.

- **Cards:** `engine/cards_v2.py` validates against the `m2radar.card.v2` shape and the closed vocabularies, then persists to `cards_v2 / card_scenes / script_versions` and writes `cards/<card_id>.json`. `engine/cards_pipeline.py` runs the whole finalisation: validate → render storyboard → persist → EDL → node validation.
- **EDL:** `engine/edl.py::card_to_edl()` produces an `m2.remotion-edl.v1` PREVIS document against `studio/remotion/contract.mjs` — your contract, ported verbatim. Scene `duration_frames` uses a cumulative-boundary rounding partition so the parts sum exactly (no `EDL_COVERAGE_INCOMPLETE / EDL_PARTITION_INVALID`). Every scene carries a `placeholder` layer, an `image` layer for the rendered template frame (`rights_approved: true`, `rights_receipt_id: "internal-template"`, sha256 of the actual PNG), and a `text` layer when `overlay_text` is non-empty. `audio_stems: []`, `pending_audio_assets: ["speech"]`, `speech_policy: "REQUIRED_RECORDED_SPEECH"` — the module never invents a speech asset.

2.7 Storyboard templates

`engine/storyboard_render.py` renders each storyboard scene to a 1080×1920 PNG with Pillow, using **only** `design/tokens/tokens.json` for colour, type and safe zones. Never a stock photo, never AI-generated imagery. Every frame is visibly a placeholder — a dashed-border box with a `[SHOT: presenter, close-up] / [SCREEN: dashboard]` label, plus the scene's `overlay_text` on an ink banner plate, a burned-in caption band from `script_text`, and a mono metadata footer. Four layouts: `a_roll`, `split_screen` (split at 58 %, matching the Remotion contract's default `split_ratio`), `demo`, `motion_graphic`. `asset_status` moves missing → template → ready → generated ; today every card scene is `template`, and the card book says so on the page.

2.8 Production contracts (Phase 3, defined and tested, not exercised)

- `schemas/supplied-take.schema.json` — the supplied-MP4 contract: `card_id`, `scene_id`, `script_segment`, `take`, `shot_type`, `roll`, `duration_s`, `fps`, `width`, `height`, `orientation`, `has_audio`, `audio_quality`, `subject`, `in_s`, `out_s`, `sync_notes`, `quality_status`, `retake_of`, `sha256`, `path`.
- `engine/production.py` — `probe_media` (ffprobe with an `ffmpeg-stderr` fallback, no new dependency, no network), `validate_take`, `ingest_takes`, `select_takes`, `edl_with_takes` (swaps only `placeholder` layers for real `video` layers; the approved storyboard is never ignored), `render_plan`. 26 tests in `tests/test_production.py`.
- `engine/storage.py` — Supabase is canonical, local is the fallback, and `schemas/render-record.schema.json` pins the render record. Cloudflare is **delivery only**: `CloudflareDelivery.public_url()` reads `CF_PUBLIC_BASE_URL` and the class never uploads anything. Both providers are `NOT_CONFIGURED` — prepared, not connected.
- Runbook: `docs/PRODUCTION_PIPELINE.md` (276 lines, commands not design).

2.9 Notion sync

`engine/notion_blocks.py` (pure block-JSON builders, no network, chunking `rich_text` at 2000 chars and block batches at 100) + `engine/notion_sync.py` (`IdCache` → `data/notion_ids.json`, `ReadOnlyTransport` with no write method at all, `LiveTransport` constructed only inside `apply_plan()`). The one non-negotiable test asserts that `build_plan()` with `discover=False` never constructs `LiveTransport`. Doc: `docs/NOTION_DASHBOARD.md`.

This was applied for real on 2026-09-12, executed from the server. What landed:

OBJECT	RESULT
Content Engine Tool (dashboard page)	body rewritten
Reels analysis	297 rows
Accounts analysis	132 rows
Cards v2	10 rows
Runs	11 rows
Insights	30 rows
Hypotheses	24 rows
Categories	276 rows

Seven databases created and filled; their ids are cached in `/opt/radar/data/notion_ids.json` (the idempotency cache, so a second run updates rather than duplicates). Three API fixes landed during the apply: child pages are kept rather than archived through `/blocks/{id}` (Notion refuses that for `child_page / child_database` — the same 400 root-caused in `engine/HANDOFF_NOTES.md`), links use `database_id` rather than a page link, and the categories scope was corrected. The last dry plan before the apply predicted Runs 7 and Categories 278; the applied numbers are 11 and 276 because the run/job tables grew and the category aggregation was re-scoped in between. Both are the truth at their own moment; the applied numbers are current.

2.10 PDF tooling

`engine/pdf_build.py` + `tools/pdf/md2pdf.mjs` — a dependency-free Markdown → PDF builder. Two documents: `reports/final/M2RADAR_ANALYTICAL_REPORT.pdf` (**132 pages, 4.26 MB**, built from 38 Markdown sections in `reports/final/partA|partB|partC/`) and `reports/final/M2RADAR_CARD_BOOK.pdf` (**122 pages, 10.17 MB**, from 12 card-book sections in `reports/final/cardbook/`). The card book was 105 pages at QA time and grew when the contact-sheet defect was fixed — see §4 of the QA note below.

2.11 Tests

Two suites on purpose (`docs/TESTING.md`, `pytest.ini`):

- **tests/** — 13 pytest modules over `engine/*`: `python3 -m pytest -q` → **252 passed, 1 skipped** (the skip is the `slow` marker in `tests/test_local_pipeline.py`). 16 warnings, all one `PendingDeprecationWarning` from `engine/charts.py:224`.
- **Root test_*.py** — 12 plain scripts, **165 checks**, all `ТЕСТ ПРОЙДЕН`. Every one now builds its own throwaway SQLite from `db.SCHEMA` instead of copying the live database. This is the fix for the 8 failures your audit-era run exposed (`reports/audit/01-branch-comparison.md` §3.7): they were fixture coupling to live data, not code defects.
- **python3 check.py** — 23/23 health checks on `data/radar.db`, including `integrity_check ok`, `video_state = 3211` distinct codes, no orphaned `beats / frame_labels / scenes`, no `jobs` row pointing at a missing run, providers registered, and spend reconciling to **\$17.78 over 889 units**.

Grand total: **441 collected, 440 passed, 1 skipped, 0 failed** locally. On the server staging copy the orchestrator reported 249 passed / 4 skipped (253 collected — the same case count, three extra skips,

no failures); that figure is recorded as supplied, not verified from this repository.

3. What was taken from Latest , what was not, and why

First, the two facts that matter most to you:

- **Your engine files were never modified.** The audit checked every shared file for which side changed it since the merge base `02102dd1`: `db.py`, `deep.py`, `collect_snapshot.py`, `blocks.py`, `score.py`, `baseline.py`, `cards.py`, `notion.py`, `notion_db.py`, `check.py`, `lib/hiker.py` are byte-identical on both branches. You built alongside main's engine, not on top of it — there were only three real collisions (`run.py`, `test_pipeline.py`, `install-cron.sh`).
- **Latest is intact.** `origin/Latest` was fast-forwarded from `31ed2d41` to `6462f409`, which is exactly your fork's `main`. Verified after the push: `origin/Latest == max/main`. The fork itself was not touched, and no work from this week was written to that branch.

FROM LATEST	TA KE N?	WHERE IT WENT / WHY NOT
Run/job state DAG (m2_orches trator/sta te.py , policy.py)	Ta ke n	engine/state.py , tables runs / jobs / video_state . Trimmed from 41 stages to the 25 this branch actually runs. Main had no state layer at all — this was the clearest gap in main.
Scene contract (schemas/v ideo- scene- segmentati on.schema. json , scene_medi a.py)	Ta ke n	schemas/ , engine/scenes.py , engine/align.py , table scenes . Main conflated cut / shot / scene; your schema separates them.
Remotion EDL contract (studio/re motion/)	Ta ke n ver bat im	studio/remotion/ , contract m2.remotion-edl.v1 , consumed by en- gine/edl.py . Main had nothing here.
Provider catalog (m2_studio /provider- catalog.js on)	Ta ke n	config/provider-catalog.json + a live registry in engine/providers.py . The catalog itself expired 2026-09-12 with price_receipt: null — the registry recomputes state from env on every run instead of freezing it.
Bounded media workers (m2_orches trator/pro cess_budge t.py)	Ta ke n	engine/process_budget.py — bounded subprocess capture, minimal env, process- group cleanup. Pure stdlib, which made it trivially safe to port.
Transcript provenance field model	Ta ke n as an ide a	transcript_meta (provider, model, asr_version, language, lang_probability, media_sha256, words, tokens, chars, speech_seconds, wps, segments_n, processing_seconds). Main recorded none of this.

FROM LATEST	TA KE N?	WHERE IT WENT / WHY NOT
PTS-indexed decode, typed failure codes (media_vis ual.py)	Partly taken	The mechanics informed <code>engine/scenes.py ; deep.py 's -ss</code> seeking was left as is because it still ships and the new path supersedes it.
Editorial claim discipline (skills/m2 -script- writer)	Take as a rule set	Claim states <code>OBSERVED / PLANNED / TO_MEASURE / MISSING</code> are a required field on every card claim, and the two reviewer prompts enforce "no invented numbers".
m2_signal / ledger + schema.sq l	Not taken	One snapshot in <code>signal.sqlite</code> is 24 MB against main's whole three-snapshot DB at 5.2 MB — 15× the storage for a read-side projection. Kept as a spec and a reference, not as a store (audit §5.3). Also: <code>legacy.py</code> discards frame evidence (2,653 frames and 282 deepdives become "frames coverage 0 %"), requires <code>done=1</code> and exactly one snapshot per date, cannot represent main's two <code>weights</code> sets, and has no field for <code>suitable / unfit_why</code> — which is what <code>cards.py</code> actually filters on.
m2_orchest rator/ 41- stage DAG	Not taken as-is	29 of the 41 stages have no code — they are agent/human handoffs. Trimmed to 25 real stages. Your own end-to-end run confirms the shape: 12 deterministic stages complete, <code>PASS_WITH_LIMITATIONS</code> , 1,535 reels, 129 accounts, and <code>final_best_reels: 0</code> by design.
m2_studio / timeline/escr ow	Partly taken	The EDL timeline idea is in <code>engine/edl.py</code> ; the provider escrow is a registry, not an escrow.
Latest's run.py	Not taken	<code>cron.sh:25</code> runs <code>.venv/bin/python run.py --yes</code> after a <code>git pull</code> . Merging your <code>run.py</code> means the next scheduled run executes the Signal planner instead of the collection pipeline — and with no <code>--yes</code> it prints <code>PLAN_ONLY</code> and exits 0, so the failure would be silent . This was named the single highest-risk collision in the audit.

FROM LATEST	TA KE N?	WHERE IT WENT / WHY NOT
Latest's install- cron.sh	No t tak en	Yours installs nothing and only prints a notice.
Latest's test_pipel ine.py	No t tak en	Both exist, both pass, they test different things. Would be <code>tests/test_signal_pipeline.py</code> on import.
Apple Vision OCR (<code>scripts/m 2_vision_o cr.swift</code>)	No t tak en	macOS-only; <code>/opt/radar</code> is Linux. There is no OCR anywhere in the merged tree.
Bare ffmpeg from PATH	No t tak en	Main uses bundled <code>imageio_ffmpeg</code> — zero system dependency. Your path needs ffmpeg on PATH, which was not even present on the audit Mac (one test skipped for it).
<code>requiremen ts-m2- media.txt</code>	No t ins tall ed	It pins 25 packages newer than what is installed and one downgrade (Pillow 12.2.0 vs 12.3.0). Installing it wholesale would disturb main's working ASR stack. Also note <code>pytest</code> is required by 22 of your 32 test modules and is in no requirements file.
<code>media_reco very.py , recovery_o verlay_rep ort.py , transcript ion_recover y.py , semantic_q ueue.py</code> (~2,900 LOC)	No t tak en	They solve problems we do not have yet. Reference only.
<code>integratio ns/radar/</code>	No t tak en	Byte-identical duplicate of the root <code>server_entry.py</code> (148 lines).

FROM LATEST	TA KE N?	WHERE IT WENT / WHY NOT
----------------	----------------	-------------------------

Your 1,062 transcripts and 19,808 frames	Could not be taken	Not in git on either branch — confirmed by a full tree scan for `transcript\` fr a .l o .j whisper across a s o u s origin/Latest and m r c t o max/main and e\ \ a p n all history. No l\ u l\ .local/ , no ts runs/20260906- \ media-acquisition- v1/`, no transcript JSONL, no frame archive. They exist only on your machine.
---	---------------------------	---

3.1 Import path if you export the ASR and frames

This is the single highest-value thing you can hand over. Preferred shape is your own `m2_signal` hash-bound receipt format so provenance survives the transfer. What the import needs on this side:

Transcripts — one row per code in `transcripts` (the existing table: `code`, `text`, `segments` as timed `[{s,e,t}]`), plus one row per code in `transcript_meta`:

```
transcript_meta(code, provider, model, asr_version, language, lang_probability,
                media_sha256, words, tokens, chars, speech_seconds, words_per_second,
                segments_n, processing_seconds, created_at, version, note)
```

Give `provider` and `asr_version` real values (ours are `provider='local'`, `model='faster-whisper/small'`, `asr_version='faster-whisper-small-int8-v1'`). Rows we could not prove provenance for are recorded as `provider='unknown'`, `asr_version=NULL` — the schema owner's standing rule is **do not "fill in" the unknown ones**, so please do not guess yours either. If your beam configs differ per batch, encode that in `asr_version` (e.g. `faster-whisper-<model>-<quant>-beam<N>-v1`) so the two corpora never get pooled by accident.

Frames — rows in `frames` (with `sha256` and `exists_ok` filled; anything that writes `frames` must set both or `refresh_video_state` treats the row as unchecked) plus `scenes` rows carrying an explicit `frames_version`. The version tag is what keeps the corpora separable: `fixed9-v1` = the legacy nine fixed samples, `aug2026` = the August archive (recognised by path `data/frames/<code>/legacy_NN.jpg` — do not rename those files), `scene-v1` = real detector output. `frames_state` only advances to `DONE_SCENE` when `scenes` rows carry `frames_version='scene-v1'`, so pick a distinct tag for yours (`m2signal-v1`, say) rather than reusing one of ours.

Then: `python3 -m engine.migrate_legacy --db data/radar.db` (re-runnable, byte-identical on a second pass) to refresh `video_state`, followed by `python3 -m engine.analyze_pending` to run `ta-v1/fa-v1` over whatever became newly eligible.

One thing has to be settled first: whether your 1,062 transcripts overlap our 275 rows, and by how much. Your own report already flags it — "a stable-ID overlap audit is still required". Do that audit on codes before the import, not after.

4. Verdict on the 7-Sep ten-card release and the 11-Sep reconciled report

Stated factually, from `reports/audit/04-existing-cards-review.md`. Nothing here is a judgement of effort; it is a judgement of what the artefacts can be used for.

4.1 What the 7-Sep slate is

`m2.editorial-scriptcard.v1`: three candidate hooks, one frozen `full_spoken_english` script (127–140 words), a `fixture` (input/output/check), 7 timed `shots` with `mode` (`a_roll/demo/motion_graphic`), `on_screen_text` and an `information_job`, matching `segments`, an `audio_plan`, a `caption_plan`, 8 assets per card, and a storyboard typed `ILLUSTRATED_CONTACT_SHEET`.

Measured facts about the ten:

- `fixture.state` is **DESIGNED_NOT_MODEL_EXECUTED on every card** — none of the "AI does X" claims was ever run through a model; they were authored as prose.
- All 80 assets are `AWAITING_OWNER_ASSET`; `manifest.json` carries `production_state: OWNER_FOOTAGE_AND_SPEECH_ALIGNMENT_PENDING` and `provider_execution: false` on every card.
- **7 of 10 are multi-source "synthesis"** (`synthesis: true` on I01, I02, I03, P01, P02, P03, P05) — an invented scenario built to illustrate a positioning point, citing no single reel.
- **0 of 10 have a statistically justified reference. 0 quote a real transcript. 0 have real frames.**
- **16 distinct reels back all 10 cards**, not the 18 the release doc claims. 12 of those 16 perform at or below their own author's median; only 4 beat it, and two of those four are 156× and 322× outliers against a ~650–6,500-view median — noise by the selection method your own findings doc prescribes ("report the denominator").
- All 16 source reels come from the agent/automation-builder niche, which `POSITIONING.md` §4 places outside M2's audience.
- Against the brand-book checklist Misha built on 8 September: 0 of 10 pass. 10 of 10 name zero tools, 10 of 10 carry zero measured numbers, 10 of 10 name neither Michael nor Max on screen. Of 70 scenes, 25 are clean; 16 generic/positioning-recap, 14 unproven (all traceable to `DESIGNED_NOT_MODEL_EXECUTED`), 12 timing/structure violations (every hook runs 4–7 s against the then-current 0–3 s house rule — note that this week's data says the 0–3 s rule itself is wrong, see §6), 12 brand-voice violations.
- The two least-bad are **M2-P03** (the owner checks the source, makes the call, records it — undercut by an invented "twelve updates") and **M2-P04** (the most actionable CTA in the slate — undercut by zero evidence and by never using the technique its own cited source demonstrates).
- Also checked and worth stating once: there is **no Luna run**. `gpt-5.6-luna` / `luna-high` is a model-preference label inside `agents/m2-roles.json` and two default parameter values, and all 17 occurrences carry `model_api_enabled: false` with `execution: deterministic_or_interactive_codex`. Searched across both branches, all history and the whole project disk. Treat every "Luna"-attributed receipt as self-authored, not independent model output.

4.2 What is reusable — the schema

The **JSON schema is worth keeping and was kept**; the content inside it was not used. Concretely, these ideas of yours are now first-class fields in `m2radar.card.v2` (`engine/SPEC.md` §8, `cards/README.md`):

- **Three candidate hooks** → `hooks[]` , minimum 2 entries, each with a `closed-vocab` type and an explicit `is_question` boolean (the house rule "no question, no tease" is now checkable mechanically).
- **Timed scenes with an information job** → `storyboard[]` with `scene_id`, `idx`, `start_s`, `end_s`, `script_text`, `script_role`, `frame_type`, `layout`, `visual`, `overlay_text`, `transition`, `source_inspiration[]`, `asset_status`, `asset_path`, `editing` . Two changes from yours: the scene count is driven by the topic rather than fixed at 7, and `layout` is restricted to exactly the four values the Remotion contract accepts, so a card that validates always has a layout the EDL can carry unchanged.
- **Claim states** → `claims[].state` ∈ `OBSERVED` | `PLANNED` | `TO_MEASURE` | `MISSING` . This came out of your 11-Sep corrected-architecture proposal and is the one part of that report adopted wholesale.
- **Duration control** → `script.total_s` is a hard error outside 20–120 s and a warning outside the 50–70 s band, computed from a live word-count-to-seconds estimate rather than a fixed 60 s template.

4.3 The 11-Sep reconciled report

Its own execution receipt states that no new collection, transcription, generation or dataset replay was run ("Local dataset replay: not run because Michael's newer raw export is absent"). Its ten cards are prose sketches, not ScriptCards. Cross-checked against main's own week-of-09-10 selections, **four of the ten are verbatim reuses of main's picks** with the copy rewritten:

YOUR CARD (11 SEP)	SIGNAL CITED	MAIN'S CARD	MAIN'S NUMBER
01 — retyping numbers from a PDF	9.1× own median	<code>Dc_tsSeAjBy</code>	9.1× (2,350,035 / 257,026)
02 — deciding who each email belongs to	1.9×, 40 shares / 62 saves per 1k	<code>DcmK70a05VP</code>	1.9×, 40/62 per 1k
03 — AI does not know how your company answers	14.4× own median	<code>DdCvFdHsnj1</code>	14.4×
04 — ten add-ons do not fix an unwritten brand rule	3.6×, 57 saves/1k	<code>Dco1m0JzZ4L</code> -as- Teardown	3.6×, 57/1k

Cards 05–10 cite no signal at all — no multiplier, no share/save rate, no reel.

The diagnosis inside that report is correct and is the most useful thing in it: your local export is 100 accounts / 2,352 reels from before 1 September, and main's 10 September export (1,535 reels, 130 accounts) was never handed to you in a joinable form. `2,352` is exactly the row count of `dataset/radar.db` , so your numbers describe that snapshot and not the current 5,147-row, three-snapshot database. That gap is now closed on our side and is the reason the numbers in this handoff will not match yours.

4.4 Independent QA of *this* week's work

`reports/audit/06-final-qa.md` , an independent read-only pass, returned **PASS WITH FIXES**: traceability chains close end to end, every reference metric in every card reproduces from `video_features` and `reels` to the digit, the scripts share **zero 5-grams** with any of the 266 transcripts, positioning filters are clean, both suites pass, no secret anywhere.

It raised five blockers and thirteen should-fixes. **All were fixed afterwards.** For the record: BL-1 a footer masking the last body line on 55 pages across both PDFs; BL-2 the card book never rendering the contact sheet and printing a local filesystem path instead (visible today as the card book at 122 pages rather than the audited 105); BL-3 "nine of the thirteen RELIABLE correlations are save-rate" — it is eleven; BL-4 a false statement about `deepdives.suitable` contradicted by card C-05 in the same deliverable; BL-5 `PRODUCTION.md` still carrying three figures this analysis disproves (now handled by `docs/PRODUCTION_REPRODUCTION_2026-09-12.md` rather than by editing an approved document in place). SF-4 was "the execution report does not exist" — it does now, at `reports/M2RADAR_INTEGRATION_EXECUTION_REPORT.md`.

5. Corpus and evidence — the denominators, and what they cannot answer

Anything you read downstream has to be divided by the right number. There are five, all from `python3 -m engine.corpus`:

NAME	N	WHAT IT COUNTS
N_ingested	3,211	reel codes with Hiker metadata — the only denominator for metric-only stats
N_transcript	275	codes with a <code>transcripts</code> row (7 of them empty)
N_transcript_usable	268	words > 0 and timed segments — the denominator for every language statement
N_frames	297	codes with <code>frames</code> rows (9 fixed samples each, <code>fixed9-v1</code>)
N_ready	268	usable transcript and frames — the denominator for script and visual conclusions

Tiers: ANALYSIS_READY 268 (8.3 %), FRAMES_ONLY 22 (0.7 %), TRANSCRIPT_UNUSABLE 7 (0.2 %), INGESTED_NOT_ANALYZED 2,914 (90.8 %). Plus 6 orphan codes that have analysis but no `reels` row and are excluded from every tier and every denominator: `DT0fJmxjVnI`, `DU90IJUCBQz`, `DZw4aTtzJpg`, `Db9AE5upfu`, `Db0UP90PDAQ`, `DcdbJKmxZ5o`.

Five things to hold onto:

- Selection bias.** `deep.py` only ever processed reels `score.py` had already ranked near the top of their creator's output. Median `robust_z` inside the analysis-ready tier is **+1.26** against **0.00** across all 3,211. Every contrast in this analysis compares strong reels with other strong reels. Absent contrasts are not evidence a feature is irrelevant; present contrasts are probably understated.
- Nine fixed frames.** The legacy visual corpus is 9 samples per reel (`fixed9-v1`). Shares (`a_roll_share` , `screen_share` , `split_share`) are estimates from those samples, and a transition between two samples is `unknown` unless the frames obviously differ. Nothing in the visual analysis may claim a transition that was not observed.
- The cut metric is a count, not a rate.** `cut_metric_quality = ffmpeg_scene_0.35_count_only` is a scene-score threshold count over 282 codes with no timestamps. It is a relative "busy vs calm" signal inside this dataset only and must never be quoted as an edit rate. This is why "cut faster" is one of the two rules the data explicitly cannot support.

4. **Media is not retained.** `local_pipeline` deletes the mp4 after processing unless `--keep-video`. Instagram's signed CDN URLs expire in hours (`docs/data-lifecycle.md` §2). So a re-analysis of an old code is a fresh paid fetch, not a local replay.
5. **Damage is logged, not hidden — ASR included.** 3,006 of 3,211 codes carry at least one flag: `MISSING_TRANSCRIPT` 2,936, `MISSING_FRAMES` 2,914, `STALE_METRICS` 1,676, `DUPLICATE_VIDEO` 56, `UNALIGNED_TRANSCRIPT` 7, `BROKEN_FRAME_REFERENCE` 1, `MISSING_MEDIA` 1, `MISSING_STATS` 1, `UNRESOLVED_CREATOR_ID` 1. Seven transcripts exist but are empty (no speech detected) and are excluded from every language statement. Eight frame files are broken on one code, `DcxV37-CJ0C` — a 0.3 MB download of a 61 s reel, 1 frame of 9 on disk — flagged `exists_ok=0` and reported by `check.py` as a known limitation, not as a discrepancy. One of the 266 transcript analyses has zero words across its beats. And a real mis-transcription of a product name is load-bearing enough to be the subject of card C-02.

1,312 reels are waiting. The newest snapshot (`snapshot_id = 3`) holds 1,535 codes, of which **1,312 are unprocessed** (no transcript, no frames) and 410 were first seen there. They are the watchdog's target set. On the server, `engine.watchdog --dry` currently selects 0 and reports 20 without a live URL — the cached CDN links have expired, as expected, until the next Monday collection refreshes them. The watchdog will only recover the subset whose cached `clips*.json` still resolves; the rest need a fresh paid Hiker call (§9, open decision 3).

Two more limits worth stating plainly: the `followers` table is empty, so **no creator-growth claim is possible**; and `our_posts` / `our_metrics` are empty, so **no claim about M2's own publishing performance is possible**. Zero of the 33 reviewed cards has ever been shot.

Finally, three numbers that all mean different things and are not interchangeable: **266** is the count of ta-v1 analysis files on disk, **268** is the count of codes the database qualifies as analysis-ready (the two extras are `DRXZJeHiAES` and `Db5sXEAP6C4`, August-archive imports with both a transcript and frames but no ta-v1 file), and **264** is the count of codes with parsed beats. And two corpus-tier vocabularies coexist deliberately — `engine.corpus.tiers()` (the *evidence* tier a report may divide by) and `video_state.corpus_tier` (the *processing* state) disagree on `FRAMES_ONLY`, 22 vs 29, because the former gives the seven empty transcripts their own `TRANSCRIPT_UNUSABLE` tier. Never mix them inside one report.

6. Ten key findings

Confidence values are the ones stored on the `insights` rows.

1. **The corpus is a selected top slice, not a sample** — 268 of 3,211 (8.3 %), median `robust_z` +1.26 vs 0.00, 102 of 132 creators represented. (*I-01, RELIABLE*)
2. **Duration does not predict performance.** Spearman(duration, `robust_z`) = -0.069 (p<0.001) over all 3,211; HIGH outliers run a median 46.8 s against 47.8 s for everything else. The 50–70 s band is a production convenience, not a measured advantage. (*I-02, RELIABLE*)
3. **Hook length carries no signal.** Median hook 6.7 s / 21.5 words / 13.2 % of speech (n=260); only 6 % close inside 3 s; $\rho \approx +0.06-0.08$, $p>0.2$ on all four metrics. The 0–3 s house rule is not supported by this niche's own data. (*I-03, PROBABLE*)
4. **Show-first hooks win on reach; setup-first hooks lose.** Demo/result-first (n=42, 29 creators) median `robust_z` 2.17 / `view_lift` 5.81 against 1.21 / 1.80. Identity-call + story-open + problem-call-out (n=39, 31 creators) median `robust_z` 0.67 against 1.36, $p=0.002$. Same creator, both

directions: DNA7-d3o5sQ +4.69 demo-first vs DXb8P08Dati -1.04 series-preamble. (I-05 / I-06, PROBABLE)

5. **Questions depress saves.** $\rho(\text{questions_n}, \text{save_rate}) = -0.229$ ($p=0.0003$), creator-normalised -0.162 — it survives. Only 9 % of hooks contain a question mark. (I-07, RELIABLE)
6. **Put a screen on camera — the single most reliable production instruction.** screen_share vs share_rate $\rho +0.203$ ($p=0.00085$), creator-normalised +0.129–0.285 across two related tests. Any-screen reels: share 0.0168 vs 0.0118 ($p=0.00013$), save 0.0308 vs 0.0220 ($p=0.001$). Coming back to the presenter afterwards (A>SCREEN>A, $n=39$, 32 creators) carries share 0.0213 vs 0.0140 ($p=0.00076$). (I-08 / I-09 / I-11, RELIABLE / PROBABLE)
7. **Split screen is the niche's most common opening and its weakest visual pattern.** 73 of 266 open on it; any split state ($n=99$) median robust_z 1.03 against 1.43 ($p=0.044$); the pure split sequence ($n=27$, 18 creators) median view_lift 0.94 against a corpus median of 2.03. (I-10, PROBABLE)
8. **The comment gate contaminates our own selection.** It runs in 59 % of the analysed corpus (158 of 266) and moves comment rate $\times 23$, save $\times 2.8$, share $\times 1.8$ — and reach not at all ($p=0.58$ –0.63). Those are exactly the two rates the radar ranks by. Until cta_type is held constant in selection, the radar is partly ranking DM funnels. (I-12, RELIABLE)
9. **Specificity is the one lexical lever that survives normalisation.** $\text{lex_entities_distinct}$ vs view_lift +0.161 pooled, +0.198 creator-normalised; strong quartile median 2 distinct named entities against 1 in the weak quartile ($p=0.011$). Conversely, pressure/urgency language runs the other way: urgency vs view_lift -0.164 ($p=0.007$), money-framed reels view_lift 1.19 vs 2.12. (I-13 / I-14, PROBABLE)
10. **Format is not the lever.** Of 115 numeric features, none separates the top from the bottom robust_z quartile at $p<0.01$. Of 460 correlation pairs, 13 are RELIABLE and **none** is against robust_z ; eleven of the thirteen are save-rate correlations. What the video controls is the save and the share, not the view. (I-22 / I-28 / I-30, RELIABLE)

Two more that matter strategically: **M2's declared territory is empty and weak** — the business-process topic is $n=8$ at $0.65\times$ lift against agent-building at $n=27$ and $6.63\times$ (I-25); and **most of the category grid is too thin to claim anything** — every one of 20 subtopic cells is INSUFFICIENT, 9 of 27 topics are, and only one hook type reaches the size threshold (I-30). The next processing wave needs breadth, not more features per reel.

6.1 The PRODUCTION.md numbers that did not reproduce

PRODUCTION.md is the approved shooting template and was **not** edited; the check is recorded beside it in docs/PRODUCTION_REPRODUCTION_2026-09-12.md. This is directly relevant to you because your 7-Sep cards were written against these figures.

STATEMENT IN PRODUCTION.MD	REPRODUCED ON THE CURRENT CORPUS	VERDICT
"measured on our own set of 2,352 competitor reels"	3,211 unique codes (2,352 was the first snapshot only)	outdated denominator
"winners' median is 55 s against 47 for the rest"	winners ($\text{robust_z} \geq 2$, $n=375$) 46.8 s vs rest 47.8 s	does not reproduce
"under twenty seconds: 5 % of winners, 13 % of the rest"	14.4 % of winners vs 12.4 % of the rest	direction reversed
"longer than ninety is 15 % against 8 %"	9.9 % vs 8.5 %	does not reproduce
"a share separates a winner 3.6× more reliably; a save, 2.8×" (RULES.md §7)	share 2.62× , save 2.09× ($n=2,899$ reels with both rates)	weaker than stated, direction holds
"editing is close to optional (median 0.12 cuts/s)"	cannot be re-measured — the stored cut metric has no timestamps	unavailable
50–70 s one-take template	not contradicted; duration is flat everywhere. Keep it on production economy, not on a measured advantage	keep, with the honest rationale

7. The ten cards

`total_s` , words and claim states from `cards_v2` ; scenes from `card_scenes` ; refs from the card JSON; verdicts from the two reviewer documents.

CARD	HY P.	FORMAT	TITLE	TOTAL _S	WOR DS	SCEN ES	RE FS	CLAIMS (OBS/PLAN/TO_MEASURE/M ISSING)	VERDICT
C-2026-09-12-01	H-11	M2 Builds	Nineteen of our top twenty were there on a trick	68.4	171	8	5	6 / 1 / 1 / 1	READY_WITH_NOTES
C-2026-09-12-02	H-12	M2 Builds	The wrong name in twenty-seven files	65.2	163	6	4	5 / 1 / 2 / 2	READY_WITH_NOTES
C-2026-09-12-03	H-18	M2 Builds	We built it, it worked, and we switched it off	69.6	167	7	4	5 / 1 / 1 / 1	READY_WITH_NOTES
C-2026-09-12-04	H-01	M2 Radar	Cost per run, not price per month	68.0	170	7	4	5 / 0 / 1 / 4	PRODUCTION_READY
C-2026-09-12-05	H-13	M2 Builds	The approval box we designed and never ticked	68.5	178	8	4	4 / 1 / 1 / 1	READY_WITH_NOTES
C-2026-09-12-06	H-04	M2 Radar	The pause is the feature	69.2	173	7	4	4 / 1 / 1 / 2	READY_WITH_NOTES
C-2026-09-12-07	H-20	M2 Teardown	The same number in two places	69.2	173	7	4	7 / 1 / 1 / 1	READY_WITH_NOTES
C-2026-09-12-08	H-09	M2 Radar	The paragraph that beat the agents	69.6	174	7	4	3 / 1 / 1 / 3	READY_WITH_NOTES
C-2026-09-12-09	H-05	M2 Radar	Supported is not the same as connected	68.0	170	7	4	5 / 1 / 1 / 2	READY_WITH_NOTES
C-2026-09-12-10	H-19	M2 Teardown	Twenty-three plans, nothing made	66.4	166	6	4	5 / 0 / 1 / 3	PRODUCTION_READY

Totals: **1,705 words, 682.1 seconds, 70 scenes, 41 references, 88 claims** (49 OBSERVED, 8 PLANNED, 11 TO_MEASURE, 20 MISSING). Format mix 4 Builds / 4 Radar / 2 Teardown, which is exactly the top ten by score — the quota never had to override the ranking. Every `total_s` is inside the 50–70 s band, all ten in its top third (69.6 s twice, 0.4 s from the ceiling — QA noted there is no headroom for a slower take on the day).

7.1 Where things live

- Card JSON (`m2radar.card.v2`): `cards/C-2026-09-12-01.json ... -10.json` .
- Storyboard frames: `cards/frames/` — 88 PNGs (70 scene frames + 10 contact sheets for the new cards, plus 7 frames + 1 sheet for the example card `C-2026-09-11-EX`).
- Remotion EDLs: `cards/ed1/C-2026-09-12-*.json` plus `C-2026-09-11-EX.json` , each with a `*.validation.json` beside it.
- Database: `cards_v2` (10), `card_scenes` (70), `script_versions` (20).
- Card book source: `reports/final/cardbook/` (12 Markdown sections) → `reports/final/M2RADAR_CARD_BOOK.pdf` .
- Reviews: `reports/analysis/10-script-review-A.md` , `10-script-review-B.md` .

7.2 How a card is structured

Top-level shape (`engine/SPEC.md` §8, `cards/README.md`):

```

schema, card_id, hypothesis_id, format, title,
strategy    { concept, audience, objective, positioning, pain, promise, rationale,
              filter { process, friction, ai_boundary, next_action } }    <- 4 mandatory
fields
references [ { code, username, url, function, reason,
              play, creator_median_play, view_lift, share_rate, save_rate,
              useful_transcript{beat_id,quote}, useful_frame{scene_id,what},
              transformed_how } ]
hooks       [ { text, type, is_question } ]                               <- >= 2
script      { version, sections[{role,text,words,seconds}], total_words, total_s,
              target_wps, tone, emotional_effect }
storyboard [ { scene_id, idx, start_s, end_s, script_text, script_role, frame_type,
              layout, visual, overlay_text, transition, source_inspiration,
              asset_status, asset_path, editing } ]
editing     { cut_timing, captions, emphasis, zooms, motion, assets_required }
claims      [ { text, state, source } ]
traceability { insights, hypothesis, evidence_summary, original_synthesis,
              why_better_than_generic }
review      { version, findings, revised }
remotion_edl_path, status

```

`validate()` enforces the closed vocabularies and, beyond them: all five performance numbers on every reference; at least two hooks; contiguous storyboard scenes (`idx` exactly `0..N-1`, each `start_s` following the previous `end_s` within 0.05 s, last `end_s` matching `script.total_s` within 0.5 s); every `script.sections[].role` covered by at least one scene; a closed-vocab state on every claim; `total_s` a hard error outside 20–120 s and a warning outside 50–70 s.

7.3 EDL validation and the one real render

All ten EDLs validated against `m2.remotion-edl.v1` on 2026-09-12T08:53:25–28Z, `render_mode` `PREVIS`, 30 fps, `returncode` `0`, `stdout`: `EDL_OK`, zero failures. Duration in frames: 01 → 2052, 02 → 1956, 03 → 2088, 04 → 2040, 05 → 2055, 06 → 2076, 07 → 2076, 08 → 2088, 09 → 2040, 10 → 1992.

One real render was executed on the example card, not simulated:

```

node studio/remotion/render.mjs --edl cards/edl/C-2026-09-11-EX.json --assets . \
  --output <scratch>/render-ex.mp4 --browser "/Applications/Google
Chrome.app/Contents/MacOS/Google Chrome"

```

3,433,827 bytes; `ffprobe` reports `00:01:00.05`, `h264` (High), `1080x1920` [SAR 1:1 DAR 9:16], 30 fps. Receipt schema `m2.remotion-render-receipt.v1`, `status` `RENDERED_REVIEW_REQUIRED`, `Remotion` **4.0.520**, `duration_frames` `1800`, `edl_sha256` `bfd8aa9a...`, `output_sha256` `0716c154...`, seven asset hashes, `provider_execution`: `false`, `downloaded_browser`: `false`. macOS 14 (Remotion warns it is older than macOS 15), Node 22.20, Chrome from `/Applications`, after `npm ci` in `studio/remotion/` (179 packages, `tsc --noEmit clean`). Evidence in `reports/evidence/`, including a frame at 12 s. The MP4 itself is not committed.

7.4 The reviewer findings pattern

Both reviewers worked to prompt version `sr-v1`, independent of the writer, recomputing every number from `data/radar.db`, `data/analysis/*.json`, `reports/data/*` and the per-reel transcripts — **no**

number in any card was accepted on the writer's word. Each review produces, per card: a verdict (PRODUCTION_READY / READY_WITH_NOTES / NOT_READY), a findings list, whether the script changed, the resulting `script.version` , and `total_s` after review.

Three cards arrived with a hard evidence error (05, 07, 09). Two of those (07, 09) would have been NOT_READY as written — card 07's explanation of *why* its two figures disagree was wrong, and card 09's central distinction contradicted our own audit. Both were fixed rather than dropped, because in each case the numbers were right and only their explanation was not. Card 01's nine findings were all evidence-source rewrites with no script change, which is why it is the one card still at `script.version: 1` .

The reviewers also flagged the brief itself: it cites `spec §28` and `spec §54` , and neither `SPEC.md` (ends at §11) nor `engine/SPEC.md` (ends at §9) has sections numbered that high. The "ten questions" checklist was reconstructed from the prompt's own item 9.

8. Where the system stands

LAYER	STATE	DETAIL
Weekly Hiker collection	automatic	<code>cron.sh</code> , <code>0 7 * * 1,4</code> on <code>/opt/radar</code> , 13 steps via <code>run.py --yes . run.py</code> estimate on the server: \$2.60 for 130 accounts.
Angle writing	automatic	<code>claude -p "\$(cat prompts/angles.md)"</code> inside <code>cron.sh</code> , after a successful run.
Notion push of week cards	automatic	<code>notion.py push</code> inside <code>cron.sh</code> (the legacy operational path).
Week pages (<code>niche/shoot/radar.html</code>)	automatic	<code>pages.py</code> , copied into <code>data/runs/<date>/</code> .
Watchdog catch-up	automatic, new	<code>engine.watchdog --limit 40 --yes</code> , runs regardless of <code>run.py</code> 's exit code, never pays.
Semantic analysis (<code>ta-v1 / fa-v1</code>)	automatic, new	<code>engine.analyze_pending --yes --limit 60</code> , one unattended agent per batch.
Feature rebuild	automatic, new	<code>engine.features --refresh .</code>
Provider registry refresh	automatic	<code>engine.providers refresh</code> at the top of <code>cron.sh</code> , overwriting the migration bootstrap.
Statistics (<code>engine.stats</code>)	manual	Needs numpy/pandas/scipy, deliberately not in the cron chain; guard it the same way if it is ever added.
Insights / hypotheses	manual	Written by an analytical pass, ingested via <code>engine.ingest_insights .</code>
Script writing / review	manual	Two prompts, <code>engine/prompts/script-writer.md</code> and <code>script-reviewer.md</code> .
Card finalisation	manual	<code>engine.cards_pipeline --all .</code>
PDFs	manual	<code>engine.pdf_build / engine.cardbook --pdf .</code>
Notion dashboard sync	manual, now applied once	<code>engine.notion_sync ; --apply</code> requires calling <code>apply_plan()</code> deliberately, a stray <code>--apply</code> flag on the CLI is hard-refused.
Phase 2 — full watchdog sweep of the 1,312	not started	The worker exists and is unit-tested; it has never been run against the live corpus with live URLs.
Phase 3 — owner footage → render → storage	not started	Contracts, validation, take selection, EDL binding and the render command all exist and are tested (26 cases). Zero real MP4s exist. Supabase and Cloudflare NOT_CONFIGURED.
Publishing	never automated	By design. Always a separate, explicit human action.

8.1 Server state

`/opt/radar` on `m2vps` is on `main`, deployed 2026-09-12 ~09:40 UTC by `git pull --ff-only`. The production database was backed up first to `data/archive/radar-2026-09-12-pre-engine.db` (byte-identical to the pre-deploy DB) and replaced by the engine DB (21.4 MB). `data/analysis/` (274 ta-v1, 295 fa-v1, plus the insights/hypotheses/refs JSON) and the archive frames were synced.

Verified on the server, Python 3.12: `engine.schema --check` up to date; `engine.hiker_config` reports `HIKER_KEY CONFIGURED` via `.env` and the Notion Cards/Reels/Accounts DBs OK; `engine.corpus ANALYSIS_READY 268 / N_ready 268`; `engine.watchdog --dry 0` selected and 20 without a live URL; `engine.analyze_pending --dry 0` pending; `bash -n cron.sh` ok; `claude` on PATH; `run.py estimate` \$2.60 for 130 accounts (the old chain intact); the staging test copy 249 passed / 4 skipped. DB counts on the server: runs 11, jobs 312, cards_v2 10, insights 30. Hardware: 6 CPUs, 11 GiB RAM, 187 GB free — enough headroom for faster-whisper `small` int8 on CPU and ffmpeg frame extraction, which is what the cron actually uses.

8.2 What Monday 2026-09-14, 07:00 should produce

This is the first scheduled run under the new schema, and it is the real test of the integration. Expected, in order:

1. `git pull --ff-only`, then `engine.hiker_config` prints one config line (the whole point of that step is that a broken Notion no longer shows up as scattered 404s mid-log).
2. `engine.providers refresh` overwrites the bootstrap provider rows with the live probe.
3. `run.py --yes` runs the existing 13 steps and writes a fresh snapshot — this is what refreshes the CDN URLs the watchdog depends on.
4. The angle agent, `notion.py push`, and the week pages, as before.
5. **New:** `engine.watchdog --limit 40 --yes` — up to 40 codes processed locally and free, from URLs the fresh snapshot just cached. Expect the first non-zero selection here; the current 0 is purely because every cached URL predates the expiry window.
6. **New:** `engine.analyze_pending --yes --limit 60` — batches exported and, if `claude` behaves unattended, ta-v1/fa-v1 written and ingested; `jobs` rows appear per batch either way.
7. **New:** `engine.features --refresh`.

Each of the three new stages is timeout-boxed and runs independently of the others and of `run.py`'s exit code, so a failure there cannot break the old chain. Things worth watching in `data/runs/<date>.log`: whether `analyze_pending` finds `claude` on PATH under cron's stripped environment (the log will say `NOT_CONFIGURED` if not, and the batch will be SKIPPED, not failed), and how many of the 40 watchdog candidates actually resolve to a live URL.

9. Open decisions and questions

9.1 Misha's calls (not yours, listed so you know what is unsettled)

1. **Two asks per card, or one?** `01-general-conclusions.md` §G rule 14 says one ask per CTA, but the mandatory four-part filter requires a `next_action` and the card carries a CTA. Reviewer A flagged this for every card in its batch — four of five ship with two asks. Options: drop the CTA and let `next_action` be the only ask; keep `next_action` out of the spoken script and put it in the caption / pinned comment; or accept two asks and retire rule 14.

2. **Do the four Builds cards stand, framed on our own pipeline?** H-11, H-12, H-13 and H-18 are all instrumented inside our own research pipeline, which is exactly the gap `PRODUCTION.md` names ("everything we have measured so far is our own tooling, which is not what the audience runs on a Monday"). The mitigation was to pick four *different generic work steps* — triage, extraction, sign-off, retirement — each naming its ordinary-business equivalent. Accept, or hold the Builds slots until a real business process has been taken apart.
3. **Pay to re-fetch the 1,312 unprocessed reels?** At the observed HikerAPI unit price of \$0.02 per request (889 units → \$17.78 to date), a re-fetch is on the order of **\$26** before transcription and frame extraction, which are free and local. Over the per-task ceiling, so it needs an explicit yes. The cheaper first move is to let Monday's watchdog run and count how many resolve from cache.
4. **Notion legacy labelling.** The audit named two ways to resolve the two-systems split: (a) re-point `NOTION_REELS_DB` / `NOTION_ACCOUNTS_DB` at the richer Signal schema and migrate `notion_db.py`'s property mapping — which needs real migration code and a manual copy of the hand-added `Verdict` / `What to borrow` columns that nothing in this repo writes back; or (b) keep them separate, link the existing operational databases into the visible page tree, and relabel the Signal ones as a one-off 2026-09-05 snapshot. **(b) is what shipped**, including the two orphaned "M2 Lab — Radar" stub pages linked under Legacy with a "candidate for deletion — needs sign-off" caption rather than deleted. If (a) is wanted instead, the applied dashboard has to be reworked.

9.2 Questions for you, numbered

1. **Do you agree with the canonical choice per layer?** Namely: main's SQLite as the store, main's `score.py` / `baseline.py` for scoring (age bands, focal-row exclusion, cross-snapshot baseline — your `metrics.py` includes the focal row and is honest about it), main's `topics.py` taxonomy, your state DAG, your scene contract, your Remotion contract, your transcript provenance model, your editorial claim discipline. If you think a layer went the wrong way, say which and what breaks because of it.
2. **Can you export your 1,062 transcripts and 19,808 frames?** Ideally through `m2_signal`'s own hash-bound receipt format so provenance survives. Target shape is in §3.1 above. Before the import we need the stable-ID overlap audit against our 275 transcript rows — your own report already calls for it.
3. **Do you want to own Phase 3?** That is: Remotion rendering with real owner footage, the take sidecar → validated → `ed1_with_takes` → render → receipt path, plus wiring Supabase as the canonical store (and Cloudflare as delivery only). The contracts and the tests are already there; nobody owns the execution. It is the part of the system closest to what you already built, and it is the only part that currently has zero real inputs.
4. **Review the scene-v1 detector thresholds.** `engine/scenes.py` uses `select='gt(scene,0.30)'`, `showinfo`, keyframe at scene start + 0.2 s, one mid-frame for scenes over 6 s, and hook samples at 0.4 / 1.2 / 2.4 / 4.0 s for compatibility with the legacy fixed-9. The legacy metric it replaces was a 0.35 threshold *count only*. You have looked harder at PTS-indexed decode than anyone here — are 0.30 and the keyframe offsets sane for 9:16 talking-head-plus-screen content, and is the hook sampling redundant once real cuts exist?
5. **cards.py ranking contamination by comment gates.** 59 % of the analysed corpus runs a comment keyword gate; it inflates comment rate ×23, save ×2.8, share ×1.8 and does nothing for reach. `cards.py`'s `_pool()` ranks on exactly those inflated rates, so the radar is partly selecting DM funnels rather than good content. The obvious fix is to hold `cta_type` constant in selection (rank within gate/no-gate strata, or exclude gated reels from reference selection entirely). Which do you prefer, and do you see a cheaper correction?

6. **Anything in Latest you consider must-port that was left behind?** The deliberate omissions are listed in §3: the `m2_signal` ledger as a store, the 41-stage DAG as-is, the media-recovery and semantic-queue modules (~2,900 LOC), the Apple Vision OCR, `integrations/radar/`, your `run.py` / `install-cron.sh` / `test_pipeline.py`. If one of those is load-bearing for something you know is coming, name it now rather than after the next merge.

Two smaller ones while you are in there: your gate `scripts/test_local.py` fails out of the box (it needs `pytest`, which is in no requirements file) and covers 7 of 32 test modules; and two tests fail at your tip in `scripts/run_m2_transcription_recovery.py:144`. Neither blocks anything here, but both mean your branch has not been run from a clean checkout recently.

10. How to run things

All commands from the repo root. Nothing below spends money except where it says so.

```

# --- health and tests -----
python3 check.py                # 23 health checks on data/radar.db
python3 -m pytest -q             # tests/ - 252 passed, 1 skipped
for t in baseline roster topup posts journal cards deep collect topics score pipeline
notion; do
    python3 test_${t}.py         # 165 checks across the 12 root scripts
done

# --- corpus, tiers, denominators -----
python3 -m engine.corpus         # tiers + the five denominators
python3 -m engine.schema --check # migration state
python3 -m engine.providers      # live provider probe (never prints a value)
python3 -m engine.hiker_config   # what cron.sh prints before every run

# --- catch-up and analysis (free, local) -----
python3 -m engine.watchdog --dry # preview selection, writes
nothing
python3 -m engine.watchdog --limit 40 --yes # process; never calls HikerAPI
python3 -m engine.analyze_pending --dry     # pending ta-v1 / fa-v1 counts
python3 -m engine.analyze_pending --yes --limit 60 # export batches + run the agent
python3 -m engine.features --refresh

# --- cards -----
python3 -m engine.cards_v2 validate cards/C-2026-09-12-01.json
python3 -m engine.cards_v2 list
python3 -m engine.storyboard_render cards/C-2026-09-12-01.json # rewrites the file in
place
python3 -m engine.edl cards/C-2026-09-12-01.json                # build + validate via
node
python3 -m engine.cards_pipeline --all                           #
validate→render→persist→EDL→node

# --- card book and PDFs -----
python3 -m engine.cardbook --pdf # markdown + PDF
python3 -m engine.pdf_build --title "..." --out reports/final/X.pdf
reports/final/partA/*.md

# --- Notion -----
python3 -m engine.notion_sync --dry --scope all # writes
reports/notion-sync-plan.md
python3 -m engine.notion_sync --dry --scope all --discover # + 2 read-only GETs,
needs .env
# --apply is hard-refused on the CLI by design; applying means calling
# engine.notion_sync.apply_plan(con, plan, id_cache) deliberately, after reviewing a
dry plan.

# --- Phase 3 -----
python3 -m engine.production takes <card_id> # probe + validate the take sidecar
python3 -m engine.storage status              # storage provider states

```

The only paid thing in the whole tree is the weekly Hiker collection (`run.py` / `collect_snapshot.py`), which prints its estimate and requires `--yes` . `engine/SPEC.md` §0.8 makes that a rule, not a habit.

10.1 Reading a trace end to end

Every processing step writes a `jobs` row against a `runs` row. To walk one card from its run back to the reels it borrowed from:

```
-- 1. the runs
SELECT run_id, kind, status, started_at, finished_at, summary_json
FROM runs ORDER BY started_at DESC;

-- 2. every job in one run, by stage
SELECT stage, state, entity_kind, entity_id, agent, provider, prompt_version,
       duration_s, error
FROM jobs WHERE run_id = ? ORDER BY started_at;

-- 3. one card, its hypothesis and its scenes
SELECT c.card_id, c.format, c.total_s, c.total_words, c.status,
       h.hypothesis_id, h.title, h.total_score, h.status
FROM cards_v2 c JOIN hypotheses h ON h.hypothesis_id = c.hypothesis_id
WHERE c.card_id = 'C-2026-09-12-01';

SELECT idx, scene_id, start_s, end_s, script_role, frame_type, layout,
       asset_status, asset_path
FROM card_scenes WHERE card_id = 'C-2026-09-12-01' ORDER BY idx;

-- 4. the references behind that card's hypothesis, with their real numbers
SELECT r.code, r.function, r.reason, r.performance_json, r.transformation
FROM hypothesis_refs r WHERE r.hypothesis_id = 'H-11';

-- 5. from a referenced code back to the evidence
SELECT code, corpus_tier, analysis_ready, transcript_state, frames_state,
       alignment_state, asr_version, frames_version, analysis_version, flags_json
FROM video_state WHERE code = ?;

SELECT idx, role, start_s, end_s, words, text FROM beats WHERE code = ? ORDER BY idx;
SELECT idx, t_sec, frame_type, roll, ui_present, text_overlay, overlay_text,
       is_visual_hook, is_proof_visual
FROM frame_labels WHERE code = ? ORDER BY idx;

-- 6. and the performance the reference claim rests on
SELECT code, play, creator_median_play, view_lift, share_rate, save_rate,
       robust_z, percentile_in_creator, outlier_status, features_version
FROM video_features WHERE code = ?;

-- 7. script history for a card (append-only)
SELECT version, kind, created_at, substr(findings,1,200)
FROM script_versions WHERE card_id = ? ORDER BY version;
```

The chain that matters: `runs` → `jobs` → `cards_v2` → `hypotheses` → `hypothesis_refs` → `video_state` → `beats` / `frame_labels` → `video_features` → `reels`. The QA pass walked three randomly chosen cards (seeded `random.seed(20260912)`: C-01, C-04, C-07) plus all ten shallowly and found the chain intact, with three unpopulated link columns: `hypotheses.card_id` is NULL for all 24 rows, and `cards_v2.run_id` / `cards_v2.pdf_page` are NULL for all 10. Those are writes that were never wired, not broken joins — worth closing before the table gets bigger.

11. Security

- **No secret is committed.** `git grep -nE "hiker_|HIKER|Bearer|x-access-key"` over `*.py` returns only environment-variable and header *names*; a scan for key-shaped literals (`x-access-key: <value>` , `secret_...` , `ntn_...`) over tracked files returns nothing.
- `.gitignore` blocks `.env` and `.env.*` with an explicit `!.env.example` exception. `git ls-files | grep -i '\.env'` returns exactly one path: `.env.example` . `.env` is mode `600` both locally and on the server, and the server's variable names match `.env.example` exactly.
- `.env.example` lists 15 variable names with empty values — `HIKER_KEY` , `LOORE_KEY` , `NOTION_TOKEN` , `NOTION_PAGE` , `NOTION_CARDS_DB` , `NOTION_REELS_DB` , `NOTION_ACCOUNTS_DB` , `OPENAI_API_KEY` , `HIGGSFIELD_API_KEY` , `SUPABASE_URL` , `SUPABASE_SERVICE_KEY` , `SUPABASE_BUCKET` , `CLOUDFLARE_API_TOKEN` , `CF_R2_BUCKET` , `CF_PUBLIC_BASE_URL` . One gap remains: `CF_PUBLIC_BASE_URL` is read by `CloudflareDelivery.public_url()` but the code degrades correctly without it.
- Paid API responses (`cache/` , `hiker-cache/` , `cache-free*/`), the database and the frames (`data/`) are all git-ignored, so signed Instagram CDN URLs and third-party biographical data never leave the machine through git. A side effect worth naming: **every evidence source cited by the deliverables is gitignored**, so a reader with only the repo cannot re-derive the numbers — the QA pass logged this as SF-13.
- `engine/providers.py` and `engine/notion_sync.py` report only the *presence* of a credential as a boolean; the two operational Notion database IDs are never hard-coded or printed by the module, its plan output or its doc.
- **One item to note.** During testing on 2026-09-11 a Notion token briefly appeared in one subagent's tool output. It was never written to a file and never committed, but a token rendered once should be treated as exposed. The recommendation stands: **rotate** `NOTION_TOKEN` and re-populate `.env` on the laptop and on `/opt/radar` . Misha has said this is not a concern, so it has not been rotated — recorded here so the decision is visible rather than forgotten.

Appendix — the shortest list of files to read, in order

1. `reports/M2RADAR_INTEGRATION_EXECUTION_REPORT.md` — the execution record; §2 branches, §5 architecture, §7 cards, §13 definition-of-done, §14 limitations.
2. `reports/audit/01-branch-comparison.md` — §Summary, §5 recommendation, §6 risks. This is the document that decided what of yours was taken.
3. `engine/SPEC.md` — §0 decisions, §2 schema, §4 taxonomies, §5 analysis contracts, §8 card JSON.
4. `docs/M2RADAR_CONTENT_ENGINE_ARCHITECTURE.md` — §2 flow, §3 module map, §12 Phase 3.
5. `reports/audit/04-existing-cards-review.md` — §B for your slate, "Format comparison and recommended information architecture" for what was kept from it.
6. `reports/analysis/01-general-conclusions.md` — §G the 16 testable writing rules, §H the synthesis.
7. `reports/analysis/08-hypotheses.md` — §1–4 for how the ten were chosen and what was rejected.
8. `docs/PRODUCTION_PIPELINE.md` and `cards/README.md` — if you take Phase 3.